Image Blur vs. Sharp Classification Using Traditional Neural Networks

Theoretical part

What is a Neural Network?

Why is it called a neural network? It is called a neural network because it is a network of interconnected elements. These elements were inspired from studies of biological nervous systems. In other words, neural networks are an attempt at creating machines that work in a similar way to the human brain by building these machines using components that behave like biological neurons.

NNs are brain-inspired computational models used in machine learning to recognize patterns & make decisions.These models consist of interconnected nodes or neurons that process data.

In this project, we use a traditional neural network (MLP) to classify images as blurred or sharp based on extracted image features.

Learning in neural networks follows a structured, three-stage process:

1. Input Computation: Data is fed into the network.
2. Output Generation: Based on the current parameters, the network generates an output.
3. Iterative Refinement: The network refines its output by adjusting weights and biases, gradually improving its performance on diverse tasks.

In an adaptive learning environment:

- The neural network is exposed to a simulated scenario or dataset.
- Parameters such as weights and biases are updated in response to new data or conditions.
- With each adjustment, the network's response evolves allowing it to adapt effectively to different tasks or environments.

What are the Benefits of Neural Networks?

- Adaptability. They can learn and make independent decisions.

- Parallel processing. Large networks can process multiple inputs simultaneously.
- Fault tolerance. Even if a part of the network fails, the entire network can still function.

What are the Limitations of Neural Networks?

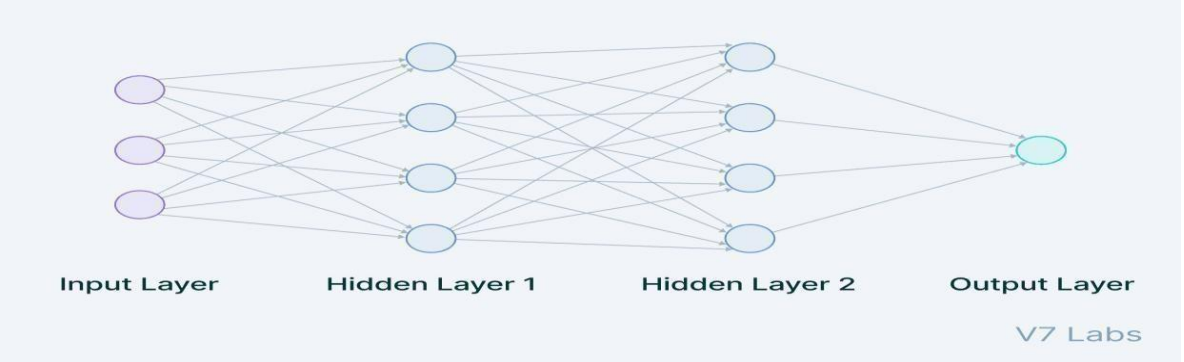
- Data dependency. They require a large amount of data to function effectively.
- Opaque nature. Often termed as "black boxes" because it's challenging to understand how they derive specific decisions.
- Overfitting. They can sometimes memorize data rather than learning from it.

Comparison of Neural Network Architectures

Neural networks are capable of learning and identifying patterns directly from data without pre-defined rules. These networks are built from several key components:

- Neurons: The basic units that receive inputs, each neuron is governed by a threshold and an activation function.
- Connections: Links between neurons that carry information, regulated by weights and biases.
- Weights and Biases: These parameters determine the strength and influence of connections.
- Propagation Functions: Mechanisms that help process and transfer data across layers of neurons.
- Learning Rule: The method that adjusts weights and biases over time to improve accuracy.

Multi-layer neural network architecture



Input Layer

The data that we feed to the model is loaded into the input layer from external sources like a CSV file or a web service. It is the only visible layer in the complete Neural Network architecture that passes the complete information from the outside world without any computation.

Hidden Layers

The hidden layers are what makes deep learning what it is today. They are intermediate layers that do all the computations and extract the features from the data. There can be multiple interconnected hidden layers that account for searching different hidden features in the data. In traditional MLP networks, hidden layers do not extract spatial features directly from images like CNN, instead they focus on learning abstract relationships between input values, each hidden layer transforms the input feature into a more meaningful representation that helps the model make clear and accurate predictions

Output Layer

The output layer takes input from preceding hidden layers and comes to a final prediction based on the model's learnings. It is the most important layer where we get the final result. In the case of classification/regression models, the output layer generally has a single node. However, it is completely problem-specific and dependent on the way the model was built. **Types of Neural Network Architectures :**

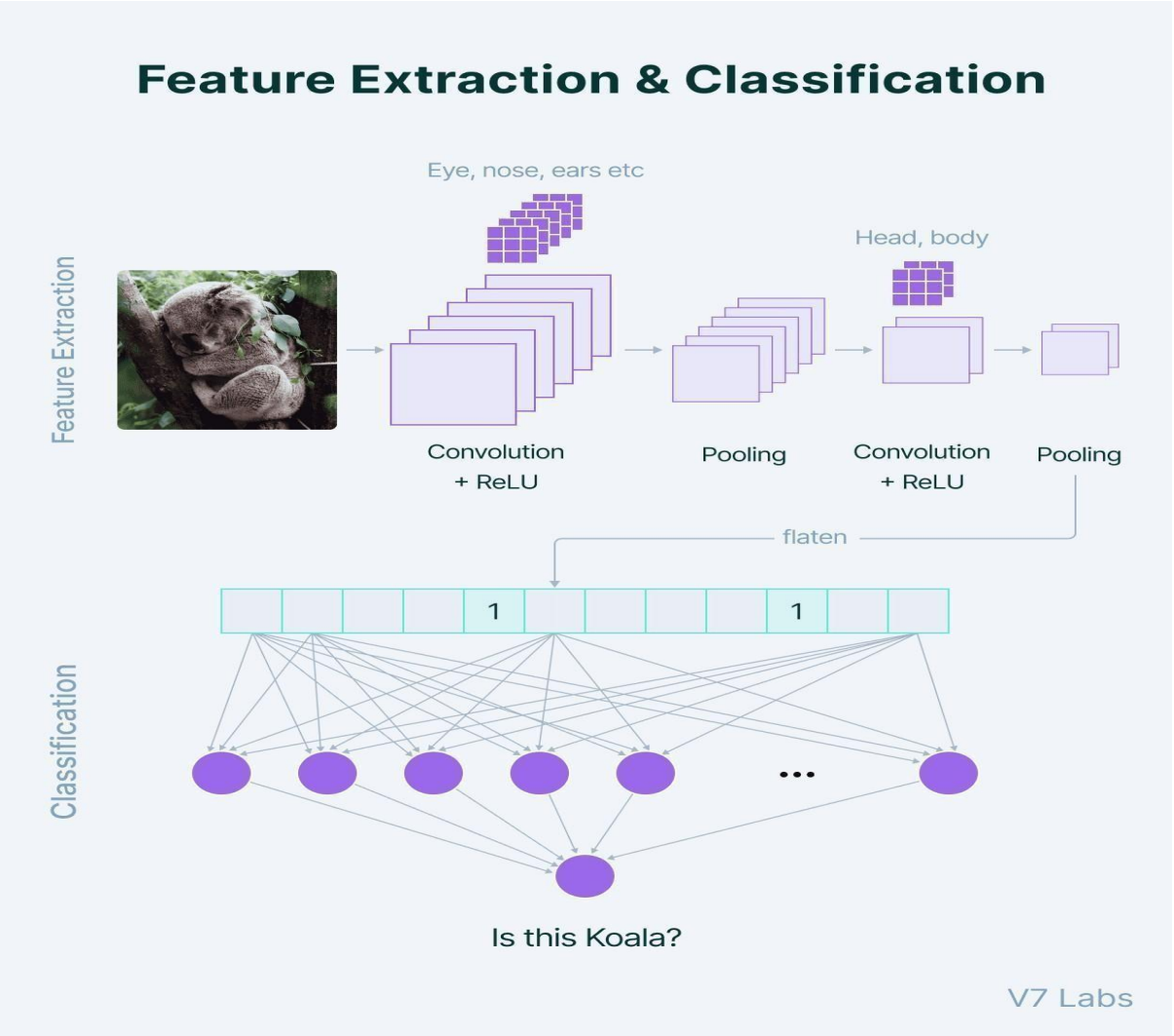
1. Standard neural networks (Traditional Feed-Forward / MLP)
2. Recurrent neural networks (RNN)
3. Convolutional neural networks (CNN)
4. Generative adversarial networks (GAN)
5. Transformer neural networks

Convolutional Neural Networks VS Traditional neural networks

Convolutional Neural Networks (CNNs)

Convolutional neural networks are designed to handle data with a grid-like topology, such as images or videos. They use a series of convolutional and pooling layers to extract features from the input data, followed by one or more fully connected layers to produce the output. The convolutional layers apply filters to the input data, detecting features such as edges and corners. The pooling layers downsample the output of the convolutional layers, reducing the spatial dimensions of the data. CNNs are often used for image classification, object detection, and other computer vision tasks.

Features refer to minute details in the image data like edges, borders, shapes, textures, objects, circles, etc. At a higher level, convolutional layers detect these patterns in the image data with the help of filters. The higher-level details are taken care of by the first few convolutional layers. The deeper the network goes, the more sophisticated the pattern searching becomes. When adding a convolutional layer to a network, we need to specify the number of filters. The value of this feature matrix is initialized with random numbers. When this convolutional layer receives pixel values of input data, the filter will convolve over each patch of the input matrix. The output of the convolutional layer is usually passed through the ReLU activation function to bring non-linearity to the model. It takes the feature map and replaces all the negative values with zero.



Traditional Neural Networks

Traditional Neural Networks are feedforward networks. They process fixed-size data with layers of neurons.They have no memory - treat each input independently. Great for classification, regression, and other simple tasks withoutSequences.

How do TNNs work?

- Input Layer: Takes in the raw data (e.g., pixel values of an image, or features of a house).
- Hidden Layers: Each neuron receives weighted sums of inputs, applies a function (activation function), and passes the result to the next layer.
- Output Layer: Produces the final result like a class label or a predicted value in regression tasks.

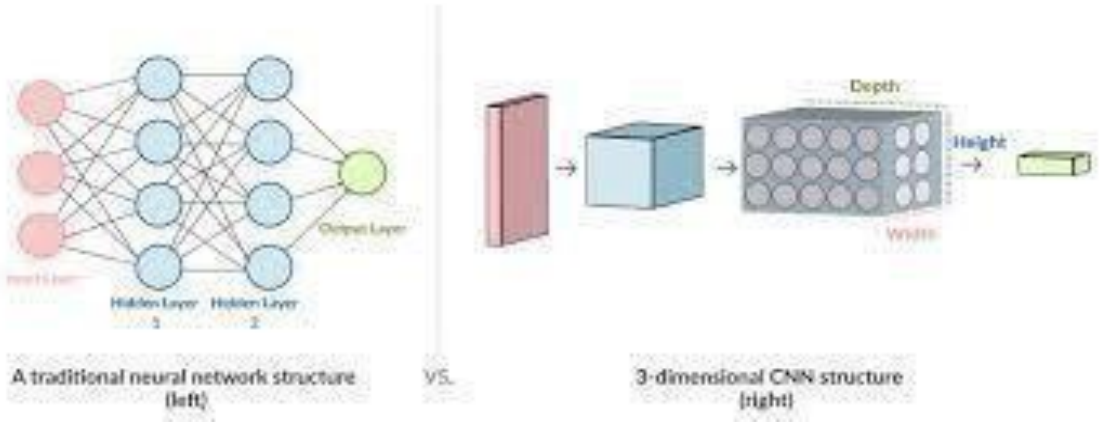
There are no loops, feedback connections, or memory mechanisms. Each input is processed independently.

Why Do We Use Traditional Neural Networks?

Traditional neural networks are especially useful when working with fixed-size input and output data. They are suitable for problems such as:

- Predicting house prices using fixed features (size, location, number of rooms)
- Classifying images with fixed-size inputs into categories (e.g., cat, dog) ● Recognizing handwritten digits (e.g., MNIST dataset)

So, Traditional neural networks (MLPs) process fixed-size input data using fully connected layers and rely on handcrafted features, treating each input independently. In contrast, convolutional neural networks (CNNs) are specialized for images and gridlike data, automatically extracting spatial features through convolution and pooling layers. This makes CNNs more effective for visual tasks, while traditional networks remain simple, lightweight, and suitable for structured data with low computational cost.



Activation functions

An activation function in a neural network is a mathematical function applied to the output of a neuron. It introduces non-linearity, enabling the model to learn and represent complex data patterns. Without it, even a deep neural network would behave like a simple linear regression model.

Activation functions decide whether a neuron should be activated based on the weighted sum of inputs and a bias term. They also make backpropagation possible by providing gradients for weight updates.Impact of Activation Functions on Model Performance What does an artificial neuron do? Simply put, it calculates a “weighted sum” of its input, adds a bias and then decides whether it should be “fired” or not

$$Y = \sum (weight * input) + bias$$

There are three type of neural network activation function :

1. **Binary step function** depends on a threshold value that decides whether a neuron activates a neuron only if input exceeds a threshold.

Limitations**:** Not suitable for multi-class tasks; zero gradient prevents effective backpropagation.

2. **The linear activation function**, Output equals input.

Limitations**:** Cannot stack layers effectively; network collapses into a single linear layer; backpropagation is limited.

3. **Non-linear activation functions** Solve the above limitations. Allow backpropagation and stacking of multiple layers.

Common types: ReLU, Leaky ReLU, Sigmoid, Tanh, Softmax, Swish, ELU, GELU, SELU.

The choice of activation function has a direct impact on the performance of a neural network in several ways:

- Convergence Speed: Functions like ReLU allow faster training by avoiding the vanishing gradient problem while Sigmoid and Tanh can slow down convergence in deep networks.
- Gradient Flow: Activation functions like ReLU ensure better gradient flow, helping deeper layers learn effectively. In contrast Sigmoid can lead to small gradients, hindering learning in deep layers.
- Model Complexity: Activation functions like Softmax allow the model to handle complex multi-class problems, whereas simpler functions like ReLU or Leaky ReLU are used for basic layers

How to choose the right Activation Function?

Hidden layers: Start with ReLU. Avoid Sigmoid/Tanh in hidden layers for deep networks.

Output layer: Depends on task type:

- Regression → Linear
- Binary classification → Sigmoid
- Multi-class classification → Softmax
- Multi-label classification → Sigmoid

Rule of thumb: Choose based on problem type and experiment for optimal performance.

NN optimization algorithms i.e. Adam and SGD

What is an optimizer?

Optimizers are algorithms or techniques used to reduce the error function (loss function) or increase production efficiency. They are mathematical functions that rely on the learnable parameters of the model, namely the weights and biases. They help determine how to change the weights and the neural network's learning rate to minimize losses. They control how the neural network's parameters are updated during training.

What is a Stochastic Gradient Descent(SGD)?

Stochastic Gradient Descent (SGD) is an iterative optimization algorithm used to minimize the loss function of a machine learning model. Unlike standard gradient descent, which computes the gradient using the entire dataset, SGD updates the model parameters using one training example at a time.

This allows efficient processing of large datasets, as the model updates its weights more frequently, leading to faster iterations. However, SGD can be sensitive to the initial values of the weights and may get stuck in a local minimum. Data normalization can help mitigate this issue, though in complex models like deep neural networks, it may not be sufficient.

Advantages :

- Efficient for large-scale data
- Performs frequent updates to model parameters
- Can converge faster per iteration, but may have higher variance than full-batch gradient descent

Use in Neural Networks:

SGD is commonly used to train neural networks, where it updates weights and biases iteratively using backpropagation to minimize the network’s loss function.

What is an Adam Optimizer (Adaptive Moment Estimation)?

Adam is an advanced optimization algorithm introduced by Diederik Kingma and Jimmy Ba in 2015. It combines the advantages of two SGD variants: momentum (first moment estimation) and RMSProp (second moment estimation).

Key features of Adam:

- Computes adaptive learning rates for each parameter individually.
- Maintains exponentially decaying averages of past gradients (first moment) and squared gradients (second moment).
- Includes bias correction to improve estimates, especially in early steps.

Advantages:

- Computationally efficient
- Requires little memory
- Works well with large datasets and high-dimensional parameter spaces

Use cases:

Adam is widely used in training deep neural networks because it adapts learning rates automatically, accelerates convergence, and handles sparse gradients efficiently.

Loss Function

A loss function is a mathematical function that is used to calculate the difference between the predicted output of the model and the actual value, during training, the loss function helps to guide the optimization process by showing how far the model's predictions are from the actual labels.

The choice of loss function depends on the type of problem , for example, the regression problems mostly use Mean Squared Error (MSE), while classification problems use different forms of crossentropy loss , selecting a suitable loss function is crucial because it impacts the learning process of the model

In this project , the Binary Crossentropy loss function was used since it is a binary classification problem (blurred vs. sharp images) . The output layer activation function is Sigmoid, which gives the probability values ranging from 0 to 1.

Binary Crossentropy loss function calculates the difference between the predicted probability and the actual target output (0 or 1) , it would penalizes incorrect predictions more strongly when the model is confident but still wrong , this makes it suitable for binary classification tasks and helps improve model performance during training.

Experiments and Results

Model Architecture

In our project, a traditional neural network architecture, also referred to as a multilayer perceptron (MLP), has been employed for classifying images as blurred or sharp, instead of passing images directly into the neural network , we have used 44 handcrafted features for each image .These features represent significant information about the images, including edge strength, texture patterns, statistical information, and frequency information, by employing these features the network is able to concentrate on important information related to the problem of blur detection while keeping computationally simple and efficient

The input layer of the network will receive the feature vector containing a 44 numerical value numbers , where each value represents a certain feature extracted from the image, these features collectively form the input representation that the neural network learns from during training.

After the input layer, the model has two hidden layers, the first hidden layer has a 64 neurons, and the activation function used for the layer is the ReLU (Rectified Linear Unit) function. The ReLU function is widely used in the construction of neural networks as it enables the complex patterns to be learned by the model without compromising the training stability, to further enhance the training stability, a batch normalization layer is used following the first dense layer, batch normalization normalizes the activations of the layer, thereby enhancing the training speed and performance of the model.

To reduce the risk of overfitting, a dropout layer with a rate of 0.3 is used after the batch normalization layer, dropout randomly disable a portion of the neurons during the training, that will force the network to learn more robust and generalizable patterns instead of memorizing the training data

The second hidden layer contain a 32 neurons ,and also used the ReLU activation function similar to the first hidden layer ,this layer is followed by batch normalization and a dropout layer with the rate of 0.2 to ensure the training stability and to improve the generalization.

Finally, the network end with an output layer that has only one neuron with a Sigmoid activation function, the sigmoid function has an output between 0 and 1, which represents the probability that the image input being one of of the two classes, an output close to 0 means the image is blurred, while an output close to 1 means the image is sharp.

The above architecture strikes a balance between simplicity and performance, as the network is simple and computationally efficient but still has the ability to learn from the image features and patterns and make accurate blur detection

Training Process

After extracting a handcrafted features from all the images, the next step was to train the neural network model, The training process included many important steps to prepare the data properly and to ensure that the model could learn meaningful patterns from the extracted features.

The features obtained from the two datasets were combined to form a single dataset, with the first dataset containing 1050 images, while the KADID-10K dataset containing 10125 images, so after the combination of the two datasets, the combined dataset contains a total of 11175 images, where each image is represented as a feature vector of size 44, representing different features of the image related to blur detection, like edges , texture, and a frequency based features.

Before training the model, the data set was split into two parts. One part will be used to train the model, while the other will be used to test the model. This was done through the train_test_split function from the Scikit-Learn library. The data was split according to a ratio of 80% for training the model and the remaining 20% for testing the model performance, this was done through stratified sampling that ensures that the class distribution between the training set and the testing set remains balanced. This will ensure that the model is able to learn properly and that the results are reliable.

After the data is split, the feature scaling is performed by the StandardScaler technique, Feature scaling is an important part of the process when handling neural networks, This is because the range of the values of the different features may be very different, for example, the range of the values of one or more of the features may be very high, while the range of the values of another or others may be very low. This would cause the model to favor some of the features simply because their values are higher, StandardScaler solves the problem by ensuring that the mean of the values of each of the features is zero and the standard deviation is one, This will stabilize the training process so that the neural network learns more efficiently.

After the data was properly prepared, the neural network model was trained using the Adam algorithm, Adam is one of the most commonly used algorithms in the deep learning because it inherits the advantages of two other algorithms , Momentum and RMSProp, It automatically adjusts the learning rate during the training process, enabling the model to converge more quickly and increasing its stability.

The loss function that was used during the training of the model was the Binary Cross Entropy, this loss function is normally used for binary classification problems , and the main goal of this project was to classify the images as either blurred or sharp. This meant that the Binary Cross Entropy loss function was used to determine how well the predictions made by the model matched the true labels. during the training of the model , the model

always tried to minimize the value of the loss function this meant that the model was improving its predictions.

The model was trained for a maximum of 50 epochs, with each epoch representing one complete pass through the training dataset, during the training of the model, the data was processed in batches of size 32, using mini batches helps speeding up the training process and allowing the model to update its weights more frequently, which will usually leads to a better learning behavior.

In addition to these settings, there is two important training technique called callbacks were used to improve the training process and to prevent overfitting.

The first callback function is EarlyStopping, This technique is used to monitor the validation loss during the training process, It stops the training process automatically if the model is no longer improving, in this project, the training is stopped if the validation loss is not improving for the last five epochs, when the training is stopped, the best weights achieved are automatically restored, this will ensure that the best model is the one being used at the end of the training process.

The second callback function is ReduceLROnPlateau, this method is used to monitor the validation loss, it reduces the learning rate if the performance of the model is no longer improving, if the validation loss is no longer improving for the last three epochs the learning rate is reduced by a factor of 0.5. This is done to allow the model to adjust the weights slightly, enabling it to find the best solution during the training process.

During the training process, both the training accuracy and validation accuracy were monitored, in addition, the training loss and validation loss were recorded at every epoch, these values are later used for generating graphs that show how the model performance improved during the training. Monitoring these metrics helps evaluate whether the model is learning effectively and whether overfitting is occurring .

Training Datasets

For training the model of blur detection that has been proposed, two publicly accessible image datasets have been used in this project. Using more than one dataset can improve the stability of the model because the neural network will be exposed into a wide variety of conditions, the datasets that have been used in this project include the Blur Dataset from Kaggle and the KADID 10K Image Quality Assessment Dataset.

The first dataset used is the Blur Dataset available on Kaggle. This dataset consists of images that have already been classified based on their blur conditions, and there is three main types of images in the dataset. These include sharp images, motion blurred images, and defocused blurred images. Sharp images include images that have well defined

edges, on the other hand, motion blur is caused by the movement of the camera or the object when being captured, and finally, Defocus blur occurs when the camera lens is not focusing on the object. For the purposes of the classification problem, the sharp images have been classified as 1. On the other hand, the motion blurred images and the defocused images have been classified as 0, this dataset consists of a total of 1050 images, features have been extracted from the images using the feature extraction method described earlier.

The second dataset that was used in this project is the KADID 10K dataset, this dataset is a large dataset that contains various images with different kinds of distortions, each image in this dataset has a corresponding DMOS (difference mean opinion score) value. The score of the image can be defined as the quality of the image as perceived through human assessment. This score will be high when the quality of the image is low. On the contrary, the score will be low when the quality of the image is high.

Since the goal of the project is classifying the images as either blurred or sharp, the DMOS values were converted to a binary values. This was achieved by setting a threshold of 3.0 to convert the DMOS values to binary values. Any image with a DMOS value higher than 3 was considered to have poor quality and was classified as a blurred image with class 0. Conversely, any image with a DMOS value less than or equal to 3 was considered to have acceptable quality and was classified as a sharp image with class 1, this approach was effective in integrating the images contained within the KADID 10K dataset with the binary classification approach used within the project, after processing the dataset, a total of 10125 images were used from KADID 10K.

Finally, after obtaining the handcrafted features for the two datasets, the feature vectors were combined into a single dataset, enabling the training of the neural network model based on a large and diverse dataset of images. The combined dataset contained a total of 11175 images, with each described by a feature vector of dimensionality equal to 44, training the neural network model based on a large and diverse dataset enables it to identify more general patterns related to image blur.

Results

Once the neural network model is trained using the combined data set, the performance of the model is evaluated using a various metrics such as overall accuracy, classification report, confusion matrix, and training curves, these tools help in measuring the performance of the model in distinguishing blurred images from sharp images

The final data set used in the training and the evaluation consists of 11175 images, and data sets were split using an 80/20 split to separate the data into a set and test set. The test set consists of 2235 images, which are used to test the performance of the model

The final training accuracy achieved by the model was 77.33% , whereas the accuracy level for the validation (testing) process is 77.99%, which is close to 78%. It is clear that there is a close relationship between the training accuracy and validation accuracy levels, indicating that the model is performing well in the unseen data, it also indicates that the model is not experiencing any overfitting issues, meaning it is learning useful patterns instead of memorizing the trained data.

In order to further evaluate the performance of this classifier, a classification report is generated, it contains precision, recall, and F1 score values for each class in the dataset.

For the blur (class 0), the model has a precision of 0.73, recall of 0.90, and an F1 score of 0.81, since the recall value for the blur class is high, it implies that the model is very good with identifying images with blur, that means that most images with blur are correctly classified by the model

For the sharp (class 1), the model got a precision of 0.87 , recall of 0.65, and an F1score of 0.74. Since the precision value for the sharp class is high, it's implying that the model is good at identifying images with sharp focus, most images with sharp focus are correctly classified by the model, however, the recall value for the sharp class is is lower, implying that some images with sharp focus were misclassified as images with blur

To better understand the nature of the classification errors made by the model, a confusion matrix is used, confusion matrix is a summary table that is used to evaluate the performance of the classification model

The model was able to classify correctly 1033 blurred images as blurred, which is referred to as True Negatives, the model was also able to classify correctly 710 sharp images as sharp, which is referred to as True Positives, there were cases when the model was not able to make the right predictions. For instance, the model was not able to classify correctly with 110 blurred images as sharp, which is referred to as False Positives, also a 382 sharp images were classified as blurred, which is referred to as False Negatives

With the analysis above, it can be noted that the model works well in the classification for the blurred images, but the model sometimes tends to classify sharp images as blurred, this can be attributed to the nature of blur detection models as the edges of blurred images are less defined compared to those of sharp images that may have different levels of texture, that makes it easier to detect the blur images

Besides the numerical values, two graphs were plotted for a better understanding of the learning behavior of the model during the training process, in the first graph, the training accuracy and validation accuracy are plotted across the training epochs, and it is seen that the accuracy is initially at a lower value but is increasing rapidly during the initial epochs of training, showing that the model is learning effectively from the patterns and the features extracted, as the training progresses, the training accuracy and validation accuracy tend to settle at approximately 77-78%. Both the accuracy values are very close to each other during the entire training process, showing that the model is learning effectively without overfitting

The second graph shows a plot of the training loss and validation loss against the training epochs using the Binary cross entropy loss function, It is seen that the loss values decreased smoothly during the training process, it decreased from 0.70 to 0.45 during the training process. This smooth decrease in loss values during the training process indicates that the learning behavior is stable and the optimizer and learning rate were correctly chosen, moreover, it is seen that the validation loss is close to the training loss during the entire training process, showing that the model is not overfitting the data.

Overall, the experimental results have shown the ability of the proposed approach, which uses handcrafted image features and a traditional multilayer perceptron (MLP) model, to differentiate between the blurred and the sharp images. Although the model has performed well in identifying blurred images, the performance could potentially be improved by modifying the features and/or the neural network model.